

บทที่ 4

เอนเตอร์ไพรส์จาวาเบิน

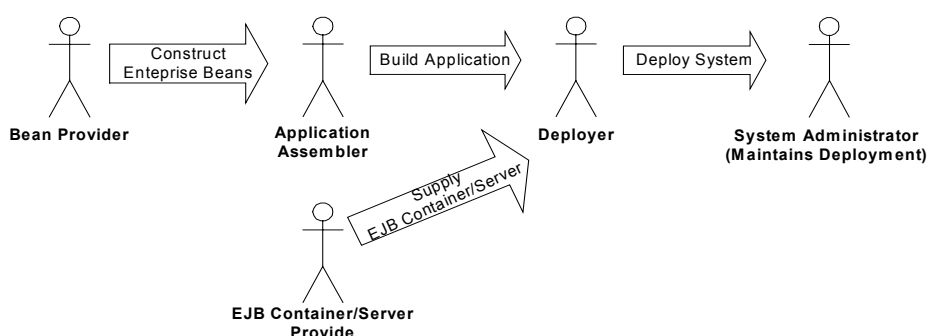
4.1 ส่วนประกอบของ EJB

Enterprise JavaBeans (EJB) ได้นำหลักการในการแบ่งการทำงานเป็นส่วน ๆ (Divide-and-Conquer) มาใช้ในการพัฒนาโปรแกรมที่ประมวลผลบนฝั่งเซิร์ฟเวอร์ EJB จะแบ่งความรับผิดชอบในการพัฒนาระบบทั้งหมดออกเป็น ส่วน ๆ โดยในแต่ละส่วนนั้นจะรับผิดชอบโดยผู้ที่มีความเชี่ยวชาญในด้านของตนเอง เวลาที่ใช้ในการดีพลอยระบบ EJB จึงลดลงมากเมื่อเปรียบเทียบกับ การพัฒนาระบบงานทั่ว ๆ ไป

หน้าที่ที่เกี่ยวข้องกับการพัฒนาระบบด้วย EJB มีดังนี้

- Bean Provider - เป็นผู้สร้างคอมโพเนนต์ทางธุรกิจที่สามารถนำกลับมาใช้ใหม่ได้ (Reusable) ซึ่งธุรกิจอื่น ๆ สามารถหาซื้อคอมโพเนนต์ไปใช้ในธุรกิจของตนเองได้ bean นั้นไม่ใช่แอปพลิเคชันที่สมบูรณ์ในตัวเอง แต่เป็นคอมโพเนนต์ซึ่งสามารถนำไปดีพลอย และประกอบกันขึ้นมาเป็นแอปพลิเคชันที่สมบูรณ์ได้
- Container Provider - เป็นผู้พัฒนาคอนเทนเนอร์ที่จะทำหน้าที่ในการจัดการกับเอ็นไวรอนเมนต์ในการทำงานให้กับ EJB
- Server Provider - เป็นผู้พัฒนาแอปพลิเคชันเซิร์ฟเวอร์ แอปพลิเคชันเซิร์ฟเวอร์คือเซิร์ฟเวอร์ในมิดเดิลเทียร์ ทำหน้าที่ในการจัดการและดีพลอยคอมโพเนนต์ต่าง ๆ โดยในขณะนี้ยังไม่มีข้อแตกต่างที่เห็นได้ชัดเจนระหว่าง Container Provider และ Server Provider
- Application Assembler - เป็นผู้ทำหน้าที่รับผิดชอบในการนำคอมโพเนนต์ต่าง ๆ มาประกอบกัน และเขียนแอปพลิเคชันที่ใช้คอมโพเนนต์เหล่านั้น โดยที่อาจจะต้องเขียนคอมโพเนนต์บางตัวขึ้นมาเอง Application Assembler อาจเป็นบริษัทภายนอกที่รับทำงานนี้โดยเฉพาะ หรืออาจจะเป็นโปรแกรมเมอร์ภายในบริษัทเองก็ได้
- Deployer - เป็นผู้นำเอาคอมโพเนนต์ต่าง ๆ ที่ได้สร้างไว้แล้วมาทำการติดตั้งลงในแอปพลิเคชันเซิร์ฟเวอร์เพื่อใช้งาน
- System Administrator - มีหน้าที่ดูแลระบบที่ถูกดีพลอยเรียบร้อยแล้วให้ทำงานเป็นปกติ

ความสัมพันธ์ในการทำงานร่วมกันของแต่ละส่วน แสดงในรูปที่ 4-1



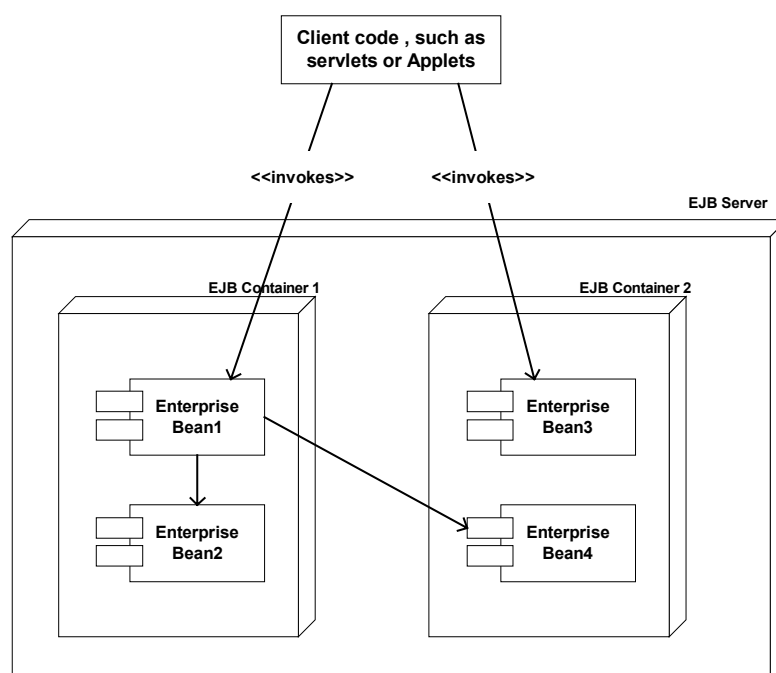
รูปที่ 4-1 ความสัมพันธ์ของแต่ละหน้าที่ในการพัฒนา EJB

4.2 EJB เซิร์ฟเวอร์ และ EJB คอนเทนเนอร์

แอปพลิเคชันเซิร์ฟเวอร์จะจัดหาบริการมิดเดิลแวร์ (middleware service) ให้กับแอปพลิเคชัน เช่น ทรานแซกชัน , ความปลอดภัย และอื่น ๆ บริการเหล่านี้จำเป็นสำหรับแอปพลิเคชันที่ต้องการความสามารถในการเพิ่มขยาย , มีประสิทธิภาพ และมีความปลอดภัย ในการให้บริการกับผู้ใช้จำนวนมากพร้อม ๆ กัน แอปพลิเคชันเซิร์ฟเวอร์จะแบ่งเป็น 2 ส่วนคือ

- EJB คอนเทนเนอร์ – เป็นที่อยู่ที่เอนเตอร์ไพรส์บีนสามารถทำงานได้ ภายในคอนเทนเนอร์หนึ่งสามารถมีบีนจำนวนมากทำงานอยู่ คอนเทนเนอร์มีหน้าที่ในการควบคุมบีนที่รันอยู่ภายในตัวมัน

- EJB Server – EJB เซิร์ฟเวอร์จะจัดหารันไทม์เอ็นไวรอนเมนต์ให้กับคอนเทนเนอร์ โดยภายใน EJB เซิร์ฟเวอร์สามารถมีคอนเทนเนอร์ได้มากกว่า 1 คอนเทนเนอร์ EJB เซิร์ฟเวอร์จะควบคุมทรัพยากรระบบในระดับล่าง และจัดสรรทรัพยากรให้กับคอนเทนเนอร์ ความสัมพันธ์ระหว่าง EJB เซิร์ฟเวอร์และคอนเทนเนอร์เป็นดังรูปที่ 4-2



รูปที่ 4-2 ความสัมพันธ์ระหว่าง EJB เซิร์ฟเวอร์และคอนเทนเนอร์

และเนื่องจาก EJB specification ไม่ได้แบ่ง EJB คอนเทนเนอร์และเซิร์ฟเวอร์อย่างชัดเจน ใน EJB specification ได้สมมติว่าทั้ง EJB คอนเทนเนอร์และเซิร์ฟเวอร์มาจากผู้ผลิตเดียวกัน และใช้ในความหมายเดียวกันคือทำหน้าที่ในการควบคุมการทำงานของ enterprise bean ตัวอย่างของแอปพลิเคชันเซิร์ฟเวอร์ที่มี EJB คอนเทนเนอร์/เซิร์ฟเวอร์ดังตารางที่ 4-1

Vendor Product	Edition Version	JDK	EJB	JSP	JMS	J2EE License	J2EE Certif.
Allaire/Macromedia JRun Server	Developer v3.1	1.2 1.3	1.1	1.1	1.0		
	Enterprise v3.1	1.2 1.3	1.1	1.1	1.0	✓	✓
ATG Dynamo	v4.0	1.2.2	1.1	1.1	1.0.2	✓	✓
BEA WebLogic	Server v6.1	1.3.1	1.1 2.0**	1.2	1.0.2	✓	✓
	Enterprise v4.1	1.2	1.1	1.1	1.0.2		✓
Borland (Formerly Inprise) AppServer	v4.5	1.2.2	1.1	1.1	1.0.2		
		1.3				✓	✓
Fujitsu Software Corp. Interstage **Limited support	Enterprise v4.0	1.2	1.1	1.1	1.0.2		
		1.3	2.0**	1.2			✓
GemStone GemStone/J	Component v4.1	1.2	1.0	1.1			
	Enterprise v4.1	1.2	1.0	1.1		✓	
	Commerce Automation v4.1	1.2	1.0	1.1			

ตารางที่ 4-1 แอปพลิเคชันเซิร์ฟเวอร์

Vendor Product	Edition Version	JDK	EJB	JSP	JMS	J2EE License	J2EE Certif.
HP Bluestone Total-e-Server	Total-e-Server v7.3	1.2.2 1.3	1.1	1.1	1.0.2	✓	✓
	Total-e-Mobile v1.1	1.2.2 1.3	1.1	1.1	1.0.2		✓
IBM Websphere	Standard v3.4.3	1.2.2		1.1	1.0.2	✓	
	Advanced v4.0	1.3	1.1	1.1	1.0.2 +XA		✓
	Advanced Single Srvr v4.0	1.3	1.1	1.1	1.0.2		✓
	Advanced Developer v4.0	1.3	1.1	1.1	1.0.2		✓
	Enterprise v3.6	1.2.2	1.0	1.0	1.0.2		
	Enterprise v4.0	1.3	1.1	1.1	1.0.2 +XA		✓
In-Q-My Application Server	v4.1	1.2.2 1.3	1.1	1.1	1.0.2	✓	✓
IONA iPortal	Standard v3.0	1.3 1.3.1	1.1	1.2	1.0.2	✓	✓
	Enterprise v1.3	1.3	1.1	1.2	1.0.2		✓
iPlanet	Enterprise v6.0	1.2.2	1.1	1.1	1.0.2	✓	✓
	Enterprise Pro v6.0	1.2.2	1.1	1.1	1.0.2		✓
JBoss **Limited support	v2.4.0	1.2.2	1.1	1.1	1.0.2		
		1.3	2.0**				

ตารางที่ 4-1 แอปพลิเคชันเซิร์ฟเวอร์ (ต่อ)

Vendor Product	Edition Version	JDK	EJB	JSP	JMS	J2EE License	J2EE Certif.
Lutris	Lutris EAS	1.3	1.1	1.1	1.0.2		
	Enhydra v3.1	1.2.2 1.3		1.1			
	Lutris Enhydra v3.4.2	1.2.2 1.3		1.1			
Oracle 9i AS	Standard v1.0.2	1.1.8 1.2.2	1.1	1.1		✓	
	Enterprise v1.0.2	1.1.8 1.2.2	1.1	1.1			
	Wireless v1.0.2	1.1.8 1.2.2	1.1	1.1			
Persistence Power Tier for J2EE	v7.0	1.2 1.3 1.3.1	1.1	1.1	1.0.2	✓	✓
Pramati Technologies Pramati Server	v2.5	1.2.2, 1.3	1.1	1.1	1.0.2		
	v3.0	1.3	2.0**	1.1	1.0.2		
**Limited support							
SilverStream Application Server	Developer v3.7.3	1.2 1.3	1.1	1.1	1.0.2	✓	✓
	Enterprise v3.7.3	1.2 1.3	1.1	1.1	1.0.2		✓
Sybase Enterprise Application Server	Small Bus. v3.6	1.2	1.1	1.1	1.0.2	✓	✓
	Advanced v3.6	1.2	1.1	1.1	1.0.2		✓
	Enterprise v3.6	1.2	1.1	1.1	1.0.2		✓

ตารางที่ 4-1 แอปพลิเคชันเซิร์ฟเวอร์ (ต่อ)

4.3 เอนเตอร์ไพรส์บีเอ็น

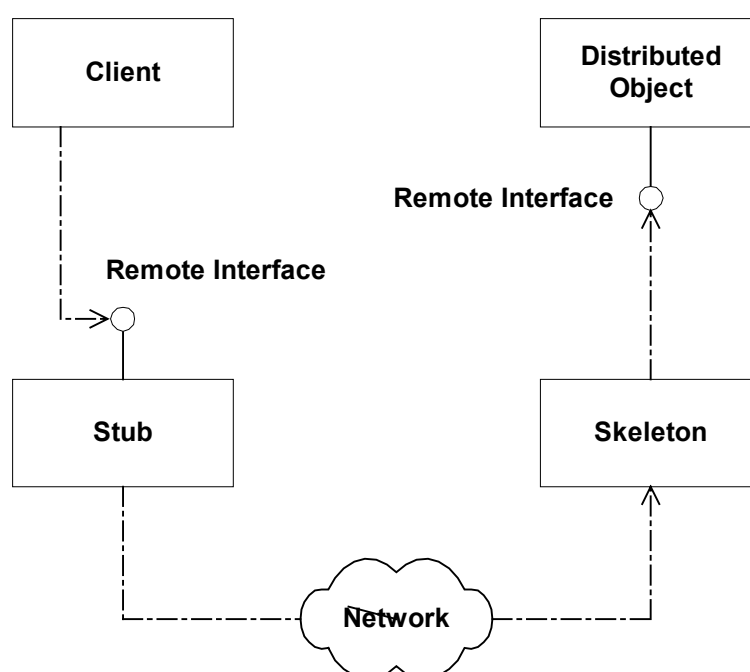
เอนเตอร์ไพรส์บีเอ็น เป็นคอมโพเนนต์ที่ทำงานบนฝั่งเซิร์ฟเวอร์ (Server-side software component) ซึ่งสามารถนำไปดีพลอยในเอ็นไวรอนเมนต์มัลติเทียร์แบบกระจาย (Distributed multi-tier environment) ได้ ภายในหนึ่งเอนเตอร์ไพรส์บีเอ็นสามารถประกอบไปด้วยจาวาออบเจกต์มากกว่าหนึ่งชิ้นได้ เนื่องจากคอมโพเนนต์นั้นไม่จำเป็นจะต้องเป็นออบเจกต์ชิ้นเดียว

ไม่ว่าเอนเตอร์ไพรส์บีเอ็นนั้นจะมีองค์ประกอบภายในอย่างไรก็ตาม ไคลเอ็นต์ที่มาเรียกใช้ เอนเตอร์ไพรส์บีเอ็นนั้นต้องติดต่อผ่านทางอินเทอร์เฟซของเอนเตอร์ไพรส์บีเอ็นเท่านั้น อินเทอร์เฟซเหล่านี้รวมทั้งเอนเตอร์ไพรส์บีเอ็นจะต้องเป็นไปตามข้อกำหนดของ Enterprise JavaBeans ข้อกำหนดเหล่านี้จะระบุถึงเมธอดที่เป็นเมธอดบังคับจำนวนหนึ่ง ซึ่งเมธอดเหล่านี้จะทำให้ EJB คอนเทนเนอร์ควบคุมเอนเตอร์ไพรส์บีเอ็นได้ในแนวทางเดียวกัน โดยไม่ขึ้นอยู่กับคอนเทนเนอร์ที่เอนเตอร์ไพรส์บีเอ็นทำงานอยู่

ไคลเอ็นต์ของเอนเตอร์ไพรส์บีเอ็นนั้นจะเป็นอะไรก็ได้ เช่น Servlet , Applet หรือเอนเตอร์ไพรส์บีเอ็นอื่น โดยในกรณีหลัง ไคลเอ็นต์ที่ทำการร้องขอไปยังบีเอ็นจะทำให้เกิดการร้องขอไปยังบีเอ็นตัวอื่น ๆ ต่อไปเรื่อย ๆ ซึ่งเป็นแนวคิดที่ดี เนื่องจากเราสามารถแบ่งบีเอ็นที่ทำงานที่ซับซ้อนเป็นส่วนย่อย ๆ ได้

4.3.1 ออบเจกต์แบบกระจาย

EJB คอมโพเนนต์มีพื้นฐานอยู่บน RMI ซึ่งเป็นเทคโนโลยีออบเจกต์แบบกระจาย รูปที่ 4-3 จะแสดงการเรียกออบเจกต์แบบกระจายของไคลเอ็นต์



รูปที่ 4-3 ออบเจกต์แบบกระจาย

- ไคลเอ็นต์เรียก stub ซึ่งเป็น proxy ออบเจกต์ทางฝั่งไคลเอ็นต์ (client-side proxy object) stub จะซ่อนการติดต่อผ่านเน็ตเวิร์ก (network communication) จากไคลเอ็นต์ stub จะรู้ว่าจะเรียกผ่านเน็ตเวิร์ก (call over network) อย่างไรผ่านทางซ็อกเก็ต
- stub จะเรียกผ่านเน็ตเวิร์กไปยัง skeleton ซึ่งเป็น proxy ออบเจกต์ทางฝั่งเซิร์ฟเวอร์ (server-side proxy object) skeleton จะซ่อนการติดต่อผ่านเน็ตเวิร์กจากออบเจกต์แบบกระจาย skeleton จะรู้ว่าจะรับการเรียกอย่างไรผ่านทางซ็อกเก็ต
- skeleton จะเป็นตัวแทนในการเรียกออบเจกต์แบบกระจาย ออบเจกต์แบบกระจายจะทำงานของมันและจะคืนผลลัพธ์ให้กับ skeleton กลับไปยัง stub ซึ่งจะคืนให้กับไคลเอ็นต์ต่อไป

จุดสำคัญคือทั้ง stub และออบเจกต์แบบกระจายจะใช้อินเทอร์เฟซเดียวกัน (ที่เรียกว่ารีโมตอินเทอร์เฟซ) ไคลเอ็นต์ที่เรียกเมธอดผ่าน stub จะคิดว่าเรียกออบเจกต์แบบกระจายโดยตรง แต่ในความจริงแล้วไคลเอ็นต์ได้เรียก stub ที่รู้ว่าจะทำมันผ่านเน็ตเวิร์กได้อย่างไร สิ่งนี้เรียกว่า local/remote transparency

4.3.2 ออบเจกต์แบบกระจายและมิดเดิลแวร์

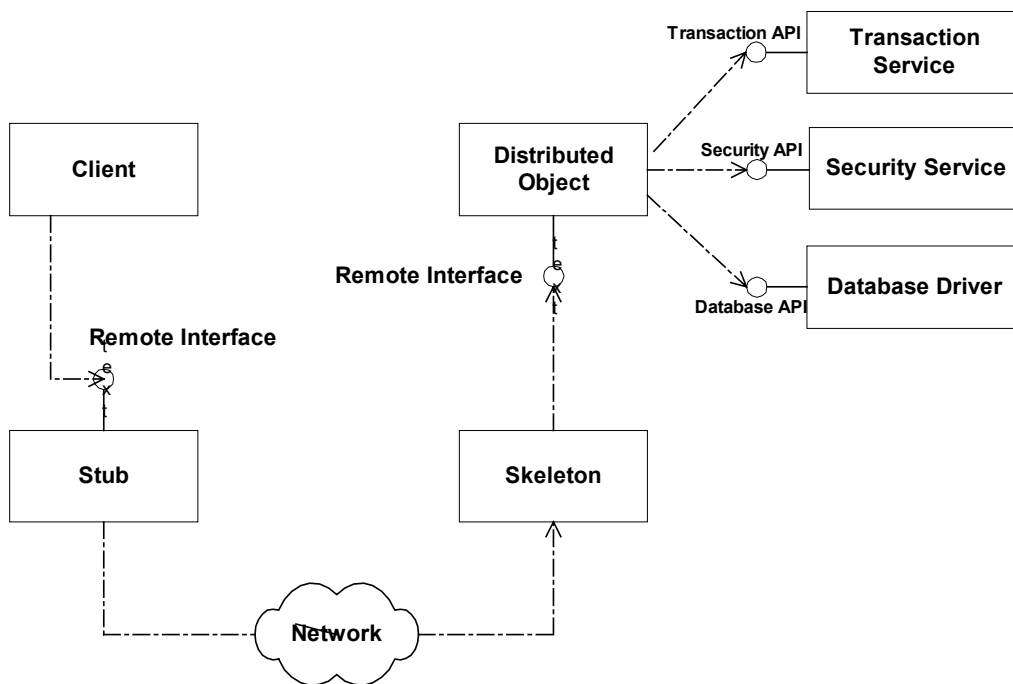
ออบเจกต์แบบกระจายนั้นมีประโยชน์มาก ทำให้สามารถเรียกใช้คอมโพเนนต์บนเน็ตเวิร์กได้ แต่มันก็ต้องการบริการจากมิดเดิลแวร์ เช่น ทรานแซกชันและความปลอดภัย

- Explicit Middleware

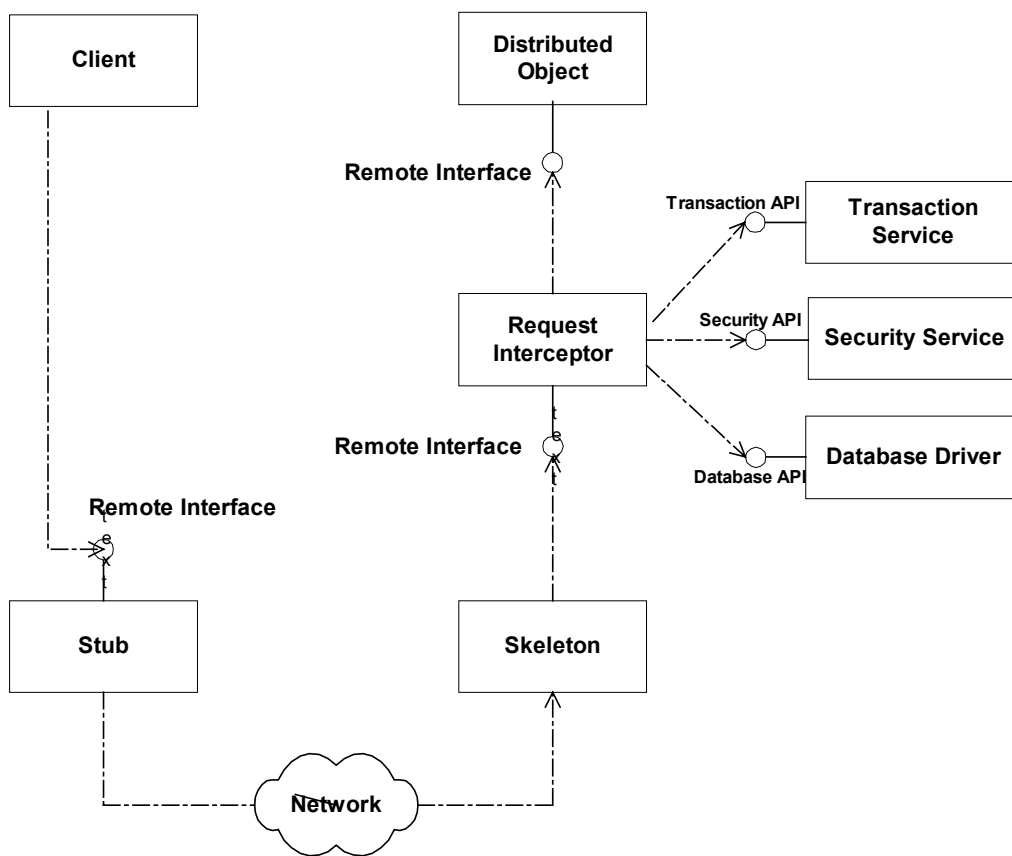
โดยปกติออบเจกต์แบบกระจายจะติดต่อกับมิดเดิลแวร์โดยเขียนโปรแกรมติดต่อกับ API ของมิดเดิลแวร์ เช่น หากต้องการการจัดการทางด้านทรานแซกชันจะต้องเขียนโค้ดติดต่อกับ transaction API ดังรูปที่ 4-4 มิดเดิลแวร์ประเภทนี้มีข้อเสียคือต้องเขียนโปรแกรมติดต่อกับ API ของมิดเดิลแวร์และหากต้องการเปลี่ยนมิดเดิลแวร์ที่ใช้ จะต้องเขียนโค้ดใหม่

- Implicit Middleware

ความแตกต่างที่สำคัญของระบบในอดีต (transaction processing monitors เช่น TUXEDO หรือ CICS หรือ เทคโนโลยีออบเจกต์แบบกระจายแบบเดิม ๆ เช่น CORBA , DCOM และ RMI) และแบบใหม่ที่เป็นเทคโนโลยีที่มีพื้นฐานจากคอมโพเนนต์ (EJB , CORBA Component Model และ Microsoft .NET) แบบนี้ไม่จำเป็นต้องเขียนมิดเดิลแวร์ API แต่จะใช้วิธีการประกาศ (declare) เรียกได้อีกอย่างว่า declarative middleware ซึ่งจะง่ายในการเขียนโปรแกรมและดูแลรักษา โดยใน EJB จะถูกประกาศใน deployment descriptor ซึ่งเป็นไฟล์ XML ที่กำหนดแท็กสำหรับการประกาศค่าต่าง ๆ ไว้



រូបទី 4-4 Explicit Middleware



រូបទី 4-5 Implicit Middleware

4.4 ชนิดของ bean (Types of Beans)

ตามข้อกำหนดของ EJB 1.0 และ 1.1 นั้น ได้กำหนดประเภทของ Enterprise Bean ไว้สองชนิดด้วยกัน คือ session bean และ entity bean และได้เพิ่ม enterprise bean อีกชนิดหนึ่งเข้าไปใน EJB 2.0 ได้แก่ message-driven bean

4.4.1 Session Bean

Session bean เป็นตัวแทนของงานที่ประมวลผลให้กับไคลเอนต์ที่เรียกใช้มัน session bean เป็นออบเจกต์ที่ใช้ในการประมวลผลลอจิกทางธุรกิจ (Business Logic) เช่น การแจ้งราคาสินค้า , การใส่ข้อมูล การส่งสินค้า , การทำธุรกรรมกับธนาคาร , การบีบอัดภาพวิดีโอ , การจัดการกับฐานข้อมูล , การประมวลผลที่ซับซ้อน และอื่น ๆ session bean จึงเป็นคอมโพเนนต์ที่สามารถนำกลับมาใช้ใหม่ได้ซึ่งบรรจุลอจิกของกระบวนการทางธุรกิจ

เหตุผลที่มันถูกเรียกว่า session bean เนื่องจากว่ามันมีอายุเท่ากับ session ของไคลเอนต์ที่เรียกใช้มัน ตัวอย่างเช่น ถ้ามีไคลเอนต์ติดต่อเข้ามายัง session bean เพื่อสั่งซื้อสินค้า แอปพลิเคชันเซิร์ฟเวอร์จะมีหน้าที่ในการสร้างอินสแตนซ์ของคอมโพเนนต์ session bean ขึ้นมา เมื่อไคลเอนต์ที่เรียกใช้ยกเลิกการติดต่อ (Disconnect) แอปพลิเคชันเซิร์ฟเวอร์สามารถทำลายอินสแตนซ์ของ session bean นั้นได้

ณ เวลาใดเวลาหนึ่ง session bean หนึ่งสามารถถูกเรียกใช้ได้จากไคลเอนต์เพียงหนึ่งตัวเท่านั้น นั่นคือ จะไม่มีการแบ่งใช้ session bean ร่วมกันระหว่างไคลเอนต์ที่เข้ามาขอใช้บริการ ซึ่งเป็นจุดที่แตกต่างจาก entity bean โดย entity bean นั้นจะถูกใช้ร่วมกันหลาย ๆ ไคลเอนต์

EJB เซิร์ฟเวอร์มีหน้าที่จัดการและควบคุมวงจรชีวิต (Life Cycle) ของทุก bean นั่นคือ ไคลเอนต์จะไม่ได้ทำการสร้างอินสแตนซ์ของ bean ขึ้นมาเองโดยตรง แต่ EJB คอนเทนเนอร์จะเป็นผู้สร้างอินสแตนซ์ของ bean ขึ้นมาให้โดยอัตโนมัติ ในทำนองเดียวกัน EJB คอนเทนเนอร์จะเป็นผู้ทำลายอินสแตนซ์ของ bean ในเวลาที่เหมาะสม ซึ่งทำให้ bean สามารถพูลและนำกลับมาใช้ใหม่กับหลาย ๆ ไคลเอนต์ที่ทำการร้องขอเข้ามาได้

session bean มี 2 ชนิดคือ stateful session bean และ stateless session bean

4.4.1.1 Stateful Session Bean

กระบวนการทางธุรกิจ (Business Process) บางอย่าง สามารถทำให้เสร็จสิ้นได้ในการเรียกใช้เมธอดเพียงครั้งเดียว เช่นการคำนวณราคาสินค้า หรือการตรวจสอบยอดบัตรเครดิต แต่กระบวนการทางธุรกิจบางอย่างไม่สามารถทำให้เสร็จสิ้นได้ในการเรียกใช้งานเมธอดเพียงครั้งเดียว

เช่นเว็บไซต์แบบอี-คอมเมิร์ซ ในการใช้งานเว็บไซต์แบบอี-คอมเมิร์ซ ลูกค้าสามารถเพิ่มรายการสินค้าเข้าไปในรถเข็นแบบออนไลน์ ซึ่งการเพิ่มสินค้าเข้าไปในรถเข็นแต่ละครั้งนั้น คอมโพเนนต์ที่รับผิดชอบจะต้องจดจำสถานะของรถเข็นได้

stateful session bean เป็น session bean ซึ่งถูกออกแบบมาเพื่อให้บริการกับกระบวนการทางธุรกิจที่ต้องมีการเรียกใช้หลายเมธอด หรือต้องมีการใช้ทรานแซกชันโดย stateful session bean จะเก็บสถานะต่าง

ๆ ของไคลเอนต์ไว้ในตัวมัน ถ้าสถานะของ stateful session bean มีการเปลี่ยนแปลงในระหว่างการเรียกใช้เมธอด ค่าสถานะนั้นจะยังคงอยู่เมื่อไคลเอนต์เดิมทำการเรียกใช้เมธอดครั้งถัดไป

4.4.1.2 Stateless Session Bean

กระบวนการทางธุรกิจบางชนิดมีรูปแบบการร้องขอบริการไปยัง bean ด้วยการร้องขอเมธอดเพียงครั้งเดียว เนื่องจากงานของกระบวนการทางธุรกิจประเภทนี้ สามารถถูกทำให้เสร็จสิ้นได้โดยการเรียกใช้เมธอดเพียงครั้งเดียว จึงไม่จำเป็นต้องมีการคงสถานะของไคลเอนต์เอาไว้ในการเรียกใช้เมธอดครั้งต่อไป โดย stateless session bean นั้นจะให้บริการกับไคลเอนต์ที่ร้องขอเข้ามาโดยไม่แบ่งแยก และจะไม่เก็บสถานะเดิมของไคลเอนต์ที่ทำการติดต่อด้วยไว้

ตัวอย่างของ stateless session bean อาจจะเป็น bean ที่รับอินพุตเพื่อใช้ในการคำนวณทางคณิตศาสตร์ที่ซับซ้อน เช่น การบีบอัดข้อมูลเสียงและวิดีโอ ไคลเอนต์สามารถส่งข้อมูลที่ยังไม่ได้ถูกบีบอัดพร้อมด้วยคุณสมบัติในการบีบอัดข้อมูลมาให้ bean ซึ่ง bean จะทำการประมวลผลข้อมูลที่ได้รับเข้ามาและส่งข้อมูลที่ถูกระบุบีบอัดแล้วกลับไปยังไคลเอนต์ หลังจากนั้น stateless session bean ตัวนี้ ก็สามารถให้บริการไคลเอนต์ตัวอื่น ๆ ต่อไปได้ สังเกตว่ากระบวนการนี้ สามารถถูกทำให้เสร็จสิ้นได้ในการเรียกใช้งานเมธอดเพียงครั้งเดียว และ bean จะไม่ต้องเก็บสถานะเดิมของไคลเอนต์ใด ๆ เอาไว้

อีกตัวอย่างหนึ่งของ stateless session bean ได้แก่ คอมโพเนนต์ที่ทำหน้าที่ตรวจสอบความถูกต้องของหมายเลขบัตรเครดิต bean ที่ทำหน้าที่นี้ จะรับข้อมูลหมายเลขบัตรเครดิต , วันหมดอายุ , ชื่อผู้ถือบัตร และจำนวนเงินเข้ามา หลังจากนั้น bean จะส่งข้อมูลตอบกลับไประบุว่า ข้อมูลนี้ถูกต้องหรือไม่โดยประมวลผลจากข้อมูลที่ได้รับเข้ามา

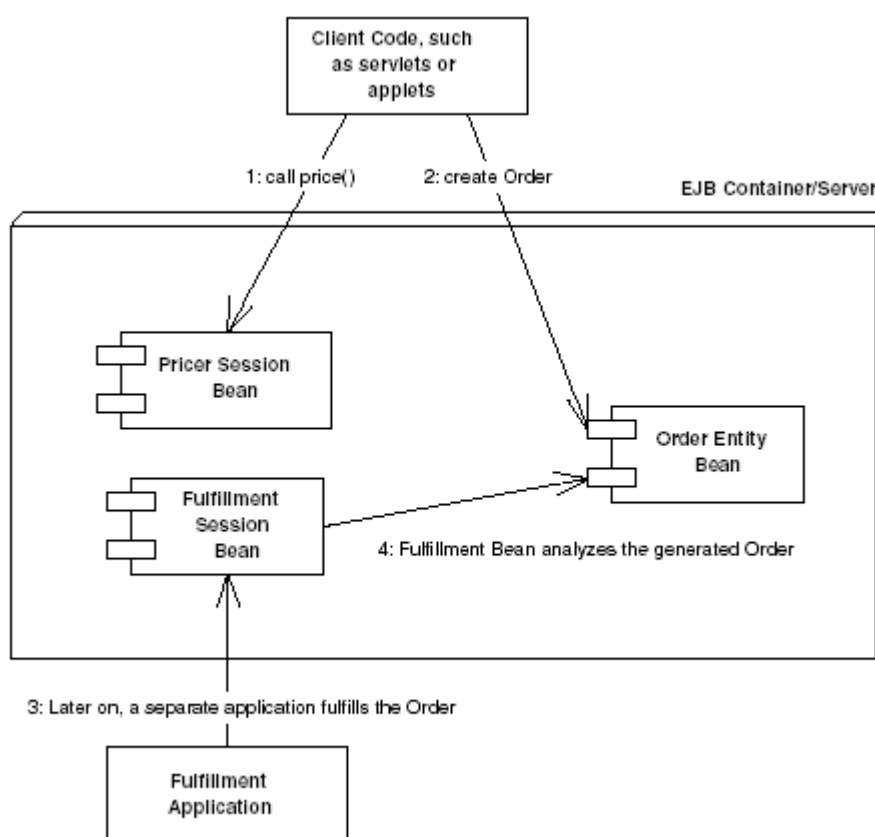
4.4.2 Entity Bean

ส่วนประกอบพื้นฐานของธุรกิจอีกส่วนหนึ่งก็คือ ข้อมูลถาวร (Permanent Data) ซึ่งกระบวนการทางธุรกิจจะต้องนำไปใช้ ดังตัวอย่างเช่น

- คอมโพเนนต์ที่ทำหน้าที่เป็นพนักงานธนาคาร จะต้องสามารถทำกระบวนการทางธุรกรรมต่าง ๆ ได้โดยข้อมูลซึ่งใช้ในการทำงานเหล่านี้ ก็คือข้อมูลบัญชีธนาคาร
- คอมโพเนนต์ซึ่งทำหน้าที่รับรายการสั่งซื้อสินค้า จะต้องทำหน้าที่บันทึกรายการสินค้าที่ถูกคำสั่งซื้อ ซึ่งข้อมูลที่ถูกสร้างขึ้นโดยคอมโพเนนต์นี้ ก็คือตัวรายการสั่งซื้อสินค้านั่นเอง ซึ่งจะต้องมีรายละเอียดของสินค้าต่าง ๆ ที่ลูกค้าสั่งซื้อ

ในแต่ละสถานการณ์ที่กล่าวมานี้ คอมโพเนนต์ของกระบวนการทางธุรกิจ ต้องเกี่ยวข้องกับข้อมูลซึ่งถูกเก็บอยู่ในแหล่งเก็บข้อมูลต่าง ๆ เช่นฐานข้อมูลเชิงสัมพันธ์ (Relational Database) เป็นต้น entity bean คือคอมโพเนนต์ซึ่งเป็นตัวแทนของข้อมูลเหล่านี้ เช่น บัญชีธนาคาร หรือรายการสั่งซื้อสินค้าดังที่ได้ยกตัวอย่างไป นอกจากนี้ entity bean อาจจะแสดงถึงออบเจกต์ของข้อมูลจริง ๆ เช่น ลูกค้า , สินค้า หรือพนักงาน

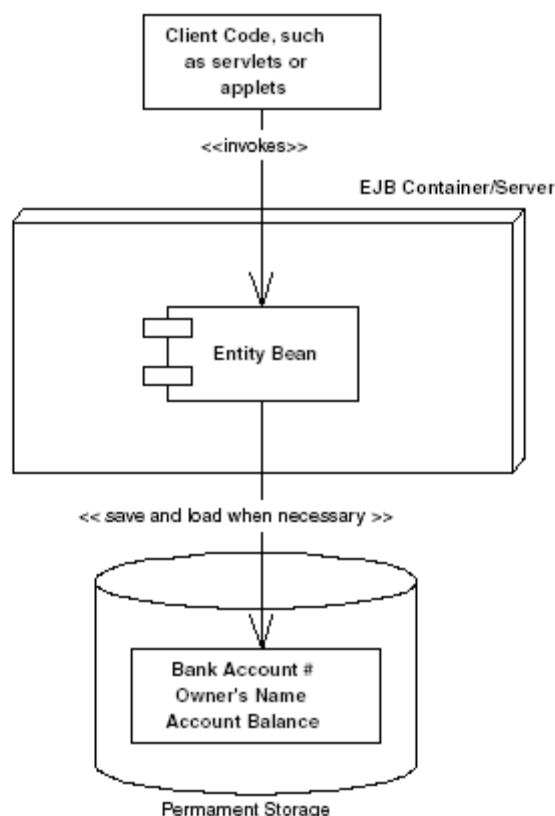
entity bean จะไม่เก็บลอจิกวิธีการประมวลผลไว้ในตัวมัน แต่จะเป็นตัวแทนของข้อมูล session bean จะจัดการกับการประมวลผลทางธุรกิจ และอาจเรียกใช้ entity bean เพื่อเป็นตัวแทนของข้อมูลที่เราต้องการติดต่อด้วย ในลักษณะเดียวกับการที่พนักงานธนาคาร ต้องใช้ข้อมูลบัญชีธนาคารในการทำงาน ดังรูปที่ 4-6



รูปที่ 4-6 การนำ Session Bean และ Entity Bean มาใช้ร่วมกันในแอปพลิเคชันการคิดราคาของสินค้าที่สั่งซื้อ

ประโยชน์ของ entity bean คือ มันจะเป็นมุมมอง (View) แบบออบเจกต์ในหน่วยความจำไปยังแหล่งเก็บข้อมูล ซึ่งถ้าเป็นวิธีการทั่วไป เมื่อแอปพลิเคชันต้องการที่จะใช้ข้อมูล จะต้องลงไปจัดการกับตารางในฐานข้อมูลเชิงสัมพันธ์โดยตรง โดยทำการอ่านและเขียนข้อมูลต่าง ๆ ที่ต้องการ ในขณะที่ entity bean จะเป็นตัวแทนของข้อมูล เราสามารถจัดการกับข้อมูลซึ่งอยู่ในฐานข้อมูลเชิงสัมพันธ์ได้เหมือนกับจัดการกับออบเจกต์ เราสามารถอ่านเซตของข้อมูลจากฐานข้อมูลมาไว้ใน entity bean ซึ่งอยู่ในหน่วยความจำ และเราสามารถจัดการกับ entity bean ซึ่งอยู่ในหน่วยความจำนี้ได้โดยการเรียกใช้เมธอดของมัน เช่น เราสามารถเรียกใช้เมธอดซึ่งมีชื่อว่า withdraw() ของ entity bean บัญชีธนาคาร เพื่อทำการลดค่าของตัวแปรแบบ private ในตัว entity bean ซึ่งมีชื่อว่า balance โดยอัตโนมัติ โดยข้อมูลจริง ๆ ซึ่งอยู่ในฐานข้อมูลจะถูกเปลี่ยนแปลงตามไปด้วยโดยอัตโนมัติ

entity bean จะช่วยให้เราใช้ความสามารถของข้อมูลซึ่งมีความถาวร (Persistent) และยังมีคุณสมบัติ encapsulation ในขณะเดียวกัน entity bean จะเป็นเลขอร์ของการเรียกใช้ข้อมูล (Data Access Logic) ในสถาปัตยกรรมแบบมัลติเทียร์ ดังแสดงในรูปที่ 4-7



รูปที่ 4-7 Entity Bean เป็นมุมมองแบบออบเจกต์ไปยังแหล่งเก็บข้อมูล

ความแตกต่างอย่างชัดเจนระหว่าง session bean กับ entity bean อีกอย่างหนึ่งก็คือ ไคลเอนต์หลาย ๆ ตัวสามารถเข้ามาใช้บริการจาก entity bean ได้พร้อม ๆ กัน ในลักษณะเดียวกับการที่ไคลเอนต์หลาย ๆ ตัวสามารถเข้าถึงข้อมูลในฐานข้อมูลได้พร้อม ๆ กัน และด้วยการใช้ทรานแซกชัน ทำให้มั่นใจได้ว่าแต่ละไคลเอนต์ที่ทำ operation จะทำงานได้ถูกต้องเป็นอิสระจากไคลเอนต์อื่น

entity bean เป็นวิธีการที่มีประสิทธิภาพเมื่อเรามีข้อมูลในฐานข้อมูลอยู่แล้ว ซึ่ง entity bean จะช่วยให้เราสามารถเข้าถึงข้อมูลเหล่านั้นได้ ข้อมูลซึ่งเรานำ entity bean มาใช้แสดงนั้น อาจมีมาก่อนที่เราจะนำ EJB มาใช้ก็ได้ ในความเป็นจริงแล้ว เรกอร์ดในฐานข้อมูลซึ่งแสดงถึงออบเจกต์หนึ่ง อาจมีมาก่อนที่ทางหน่วยงานจะตัดสินใจนำภาษาจาวามาใช้ในการพัฒนาแอปพลิเคชัน เนื่องจากว่าโครงสร้างของฐานข้อมูลนั้นไม่ขึ้นกับภาษาใด ๆ (Language-Independent) เรกอร์ดในฐานข้อมูลสามารถที่จะแปลงเป็นออบเจกต์ได้ในเกือบจะทุกภาษา ซึ่ง EJB ได้ใช้ประโยชน์ตรงจุดนี้ โดยอนุญาตให้มีการแปลงข้อมูลในฐานข้อมูลให้

เป็นออบเจกต์ในภาษาจาวา กระบวนการแปลงข้อมูลนี้ เราอาจจะจัดการด้วย bean หรือให้คอนเทนเนอร์จัดการให้โดยอัตโนมัติก็ได้

entity bean สามารถแบ่งออกได้เป็นอีกสองประเภท ได้แก่ entity bean แบบ bean-managed persistent และแบบ container-managed persistent

4.4.2.1 Bean-managed Persistent Entity Bean

จากที่กล่าวมาแล้ว entity bean เป็นคอมโพเนนต์ที่มีคุณสมบัติ persistent เพราะว่าสถานะของมันนั้นจะถูกบันทึกลงในแหล่งบันทึกข้อมูล (Secondary Storage) เช่นในฐานข้อมูลเชิงสัมพันธ์ ตัวอย่างเช่น ด้วยเทคโนโลยี Object-relational Mapping เราสามารถแมปออบเจกต์ที่อยู่ในหน่วยความจำไปเป็นเรคอร์ดในฐานข้อมูลเชิงสัมพันธ์ซึ่งมีคุณสมบัติ persistent ได้ และเราสามารถนำข้อมูลนี้กลับมาใช้ได้ทุกเมื่อโดยการดึงข้อมูลในฐานข้อมูลกลับมาสร้างเป็นออบเจกต์ในหน่วยความจำอีกครั้ง

อีกแนวทางหนึ่ง คือการใช้ฐานข้อมูลแบบออบเจกต์ (Object Database) เป็นแหล่งบันทึกข้อมูลแบบ persistent ซึ่งจะบันทึกออบเจกต์จริง ๆ ลงไปแทนที่จะเป็นเรคอร์ดของความสัมพันธ์

bean-managed persistent entity bean เป็น entity bean ที่ต้องทำการเก็บข้อมูลต่าง ๆ แบบ persistent ด้วยตัวเอง นั่นคือ ผู้พัฒนาคอมโพเนนต์จะต้องเขียนโค้ดในการแปลงฟิลด์ต่าง ๆ ของ entity bean ซึ่งอยู่ในหน่วยความจำให้กลายเป็นข้อมูลซึ่งเก็บอยู่ในแหล่งบันทึกข้อมูลเช่นฐานข้อมูลเชิงสัมพันธ์ ซึ่งจะต้องเขียนโค้ดสำหรับการเขียน การอ่าน และการค้นหาข้อมูลเข้าไปเป็นส่วนหนึ่งของ entity bean โดยอาจทำผ่านทาง API แบบ persistent ต่าง ๆ เช่น JDBC หรือ SQL/J

4.4.2.2 Container-managed Persistent Entity Bean

Enterprise JavaBeans ช่วยให้นักพัฒนาสามารถพัฒนา entity bean ได้อย่างสะดวกมากขึ้น โดยไม่ต้องเสียเวลายุ่งเกี่ยวกับโค้ดในการทำ persistent ให้กับข้อมูล ตามข้อกำหนดของ EJB 1.1 คอนเทนเนอร์จะต้องสนับสนุนการ persistent ข้อมูลโดยอัตโนมัติสำหรับ entity bean

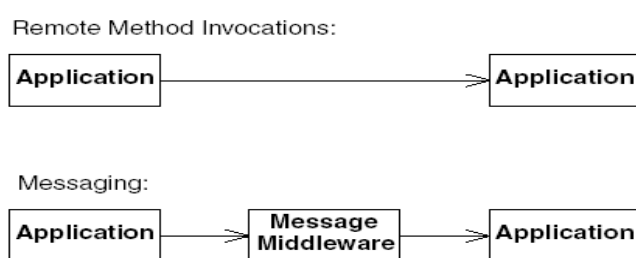
คอนเทนเนอร์จะทำฟังก์ชันในการเข้าถึงข้อมูลโดยอัตโนมัติ รวมถึงการเก็บข้อมูล การอ่านข้อมูล และการค้นหาข้อมูลของคอมโพเนนต์ โดยที่ผู้พัฒนาไม่ต้องเขียนโค้ดในการติดต่อกับฐานข้อมูลด้วยตัวเองซึ่งจะช่วยประหยัดเวลาในขั้นตอนการพัฒนาไปได้มาก สิ่งที่ผู้พัฒนาต้องทำก็คือการระบุว่าต้องการให้ฟิลด์ใดบ้างที่ต้อง persistent หลังจากนั้น คอนเทนเนอร์ก็จะดูแลการ persistent ข้อมูลให้ตามที่กำหนด

ด้วยวิธีการดังกล่าว จะทำให้เราได้โค้ดที่ไม่ขึ้นอยู่กับระบบฐานข้อมูล ทำให้เราสามารถเปลี่ยนแปลงแหล่งเก็บข้อมูลได้อย่างอิสระ เนื่องจากใน Container-managed ไม่มีการเขียนโค้ดติดต่อกับ API ของระบบฐานข้อมูล

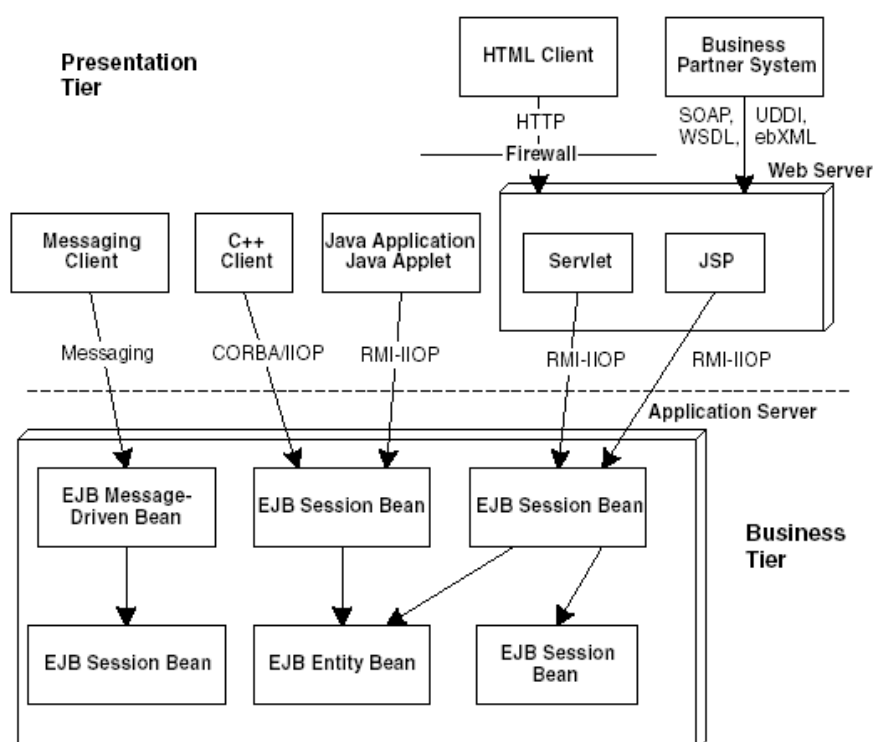
4.4.3 Message-driven bean

เป็นอีกแนวทางของรีโมตเมธอด โดยใช้เมสเซจซึ่งเป็นการติดต่อแบบ asynchronous ภายหลังจากการส่งเมสเซจออกไป ผู้สร้างเมสเซจสามารถที่จะประมวลผลงานอื่น ๆ ได้ ภายหลังจากที่ผู้รับเมสเซจประมวลผลเสร็จก็อาจกำหนดให้ส่งผลลัพธ์กลับมาให้กับผู้สร้างเมสเซจได้

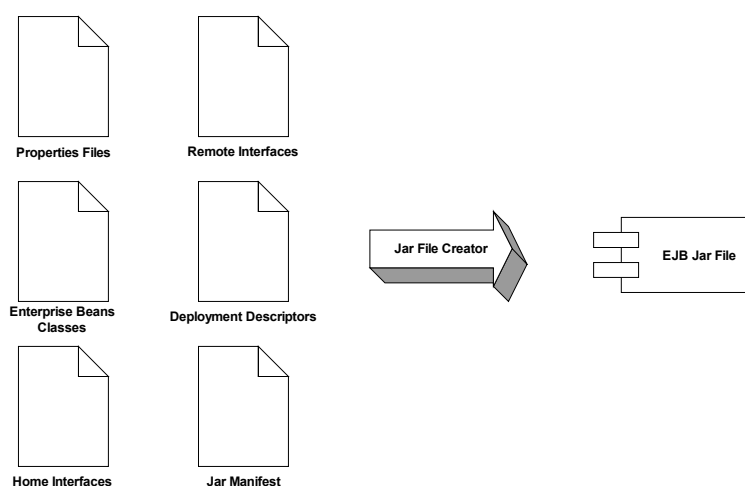
โครงสร้างพื้นฐานที่รองรับการใช้เมสเซจนี้จะเรียกว่า Message-oriented middleware (MOM) มีผลิตภัณฑ์หลายชนิดที่ใช้โครงสร้างพื้นฐานของ MOM เช่น IBM MQSeries , BEA Tuxedo/Q และ Microsoft MSMQ เป็นต้น



รูปที่ 4-8 เปรียบเทียบรีโมตเมธอดกับเมสเซจ



รูปที่ 4-9 การเรียกใช้ enterprise bean



รูปที่ 4-10 ขั้นตอนการสร้างไฟล์ Ejb-jar

หลังจากสร้างไฟล์ ejb-jar แล้ว ถือว่าเอนเตอร์ไพรส์บีนสมบูรณ์ สามารถนำไปดีพลอยในแอปพลิเคชันเซิร์ฟเวอร์ได้ เมื่อนำไปดีพลอย เครื่องมือของผู้พัฒนาคอนเทนเนอร์จะทำหน้าที่ขยายไฟล์ ejb-jar และนำข้อมูลต่าง ๆ ในไฟล์มาใช้งาน